Volume 03 - Runtime Forecasting, Feature Engineering, and Model Training

Revision: 4405d167fc323f33be76e2c7f6614fd21cc574a8 **Sources:** app/forecasting.py; pipeline/03_train_and_evaluate_forecaster.py; models/evaluation_metrics.json; metadata producer/consumer contracts. **Code excerpts:** the PDF appendix includes all functions in both Python modules and their module-level constants/import setup with original line numbers.

1. Two related but different flows

Training is an offline, heavy operation: read millions of synthetic rows, construct features and historical aggregates, fit baselines and a gradient-boosted tree model, evaluate, then serialize estimator and metadata. Runtime inference is a lightweight operation: a Flask process loads the serialized artifacts once, makes a one-row feature table for a chosen station/date/hour, asks the model for one prediction, then post-processes that result.

The large generated CSV is a training input. The running app does not read it. The app reads the estimator, metadata, cleaned station table, and small metrics file.

2. Runtime module initialization: app/forecasting.py:1-32

Imports: os/path utilities, joblib, NumPy, pandas, datetime, calendar helper and service-window predicate. BASE_DIR is computed relative to this file, then used to construct artifact paths. At import:

1. joblib.load deserializes model and metadata into process globals _MODEL and _META. 2. pandas reads stations_cleaned.csv and indexes rows by Station. 3. _METRICS starts as an empty dict; if the JSON path exists, json.load populates it.

These are startup side effects, not lazy per-request loads. Missing model/metadata or station CSV can prevent app startup. Metrics file is optional, so absent metrics yields empty saved metrics plus runtime provenance keys. joblib must be trusted and version-compatible; the code does not validate estimator feature count against metadata before serving.

3. get_station_info(station: str) -> dict

Purpose: adapt one station-directory row to the inference/API structure. Known station: split All_Lines on commas and strip whitespace, return primary line, bool interchange, and float coordinates if non-null. If All_Lines column is absent, fallback to [Line]. Unknown station: return "Unknown", no lines, false interchange and null coordinates.

Input is a station name string. Output dict fields are station, line, all_lines, is_interchange, latitude, longitude. It reads global _STATIONS_DF and does not mutate it.

Illustrative return shape: {"station":"Rajiv Chowk","line":"Blue Line","all_lines":["Blue Line","Yellow Line"],"is_interchange":true,"latitude":28.63,"longitude":77.22} Coordinate values here are illustrative; use CSV for exact values.

Edge: duplicate station index rows can make .loc return a DataFrame instead of Series; the code expects one row. Unknown station fallback prevents lookup error, but later forecasting maps missing station index to 0, and optimizer can fail because all_lines is empty. It is safer for API routes to reject unknown names than to treat them as valid forecast identities.

4. predict_hourly_footfall(station: str, dt_obj: datetime, hour: int) -> int

This is the model inference core.

Step-by-step construction

1. Hours below 5 or above 23 return 0 before any metadata lookup. 2. Get station information. Look up Station_Idx and Line_Idx in saved dictionaries; absent keys default to index 0. This keeps array/schema inputs finite but can silently map unknown categories to the first learned category. 3. Derive weekday via datetime.weekday() (Monday=0) and IsWeekend as 1 for Saturday/Sunday. 4. Encode cyclic time: sin/cos pairs for hour/24 and weekday/7. Pairs preserve wraparound similarity, e.g. Sunday and Monday are near neighbors on the weekly circle. 5. Read historical fallback features from metadata. Hist_Station_Mean falls back to global_mean. Hist_Hour_Day_Mean key is station-index + weekday + hour; missing key falls back to station mean. 6. Obtain month and call get_calendar_context(dt_obj.date()). HolidayType comes from that context; IsHoliday is true when holiday type is nonzero; IsWorkday comes from calendar context. 7. Get station typology from saved map, default RESIDENTIAL_SUBURB; map to integer typology index default 0. Get station-month historical mean, fallback to station mean. 8. Construct a dictionary of 23 named features. These include raw date parts; weekend/holiday/workday; event class; six trig values for hour/day/month; station/line/typology indexes; interchange flag and number of lines; three historical means. 9. Build a one-row pandas DataFrame with exactly the saved feature_cols order: [{name: values[name] for name in ...}]. That order is a training-serving schema contract. 10. _MODEL.predict(features)[0] extracts the first (only) prediction. Convert to float, round, then clamp below zero to zero; return integer.

Output type: int. **Side effects:** none beyond CPU/memory allocation. **Calls:** get_station_info, get_calendar_context, estimator predict. **Complexity:** feature calculation is O(1) relative to dataset size; model tree traversal depends on estimator size/depth. Building a DataFrame adds small allocation overhead.

Worked feature example (symbolic values)

Suppose the user chooses a known station at Wednesday, 14 October 2026, hour 09. Then weekday=2, IsWeekend=0, month=10, and the date-derived day fields follow datetime. Hour cyclic values are sin(2*pi*9/24) and cos(2*pi*9/24). Historical values come from serialized maps using the selected station index. The values dict is reordered by metadata feature_cols before predict. The actual encoded station id and counts require reading this checkout's binary metadata and are not invented here.

Risks and boundaries

• Direct hourly function knows only integer hours. The API service gate handles minute-level 23:30 closure.

• Unknown station gets station index zero and unknown line index zero; prediction may be numerically returned but semantically misidentified.

• Missing feature_cols entry causes KeyError before predict. A metadata feature name absent from values also causes KeyError.

• Training/inference category maps and model must be versioned together.

• Negative raw regressor output is clipped to zero after rounding.

• Calendar categories are static project scenarios, not measured event-response effects.

5. predict_footfall_with_minute_interpolation(...)

Signature: station, dt_obj, hour, minute -> integer estimate. First calls is_service_operating(hour, minute); closed times return 0. Otherwise it gets v0 at selected integer hour and v1 at min(23, hour+1), then computes v0 + (v1-v0)*(minute/60.0), rounds and clamps nonnegative.

At 09:15, weight is 15/60=.25: estimate is 0.75*v09 + 0.25*v10 before rounding. At 23:20, both v0 and v1 refer to 23:00 because next_hour is capped at 23, so interpolation remains the 23 estimate; at 23:31 the operating-hours gate returns zero. The algorithm assumes demand changes linearly between hourly model outputs. It creates smooth interpolation, not measured quarter-hour counts.

6. predict_station_full_day_profile(station, dt_obj) -> dict

Creates hours=list(range(5,24)), exactly 19 integer points. It loops over hours and appends model predictions plus corresponding training-history means. Labels format each as HH:00. np.argmax finds the first maximum index; labels[index] becomes peak_hour. sum(predicted_curve) becomes predicted_daily_total.

It then obtains station line list. For multiple lines, it imports get_interchange_line_split locally and reads a modeled share; for one line it creates 100% share. For each line, each hourly predicted value is multiplied by share and rounded. Return fields are labels, hours, predicted, historical, peak_hour, peak_footfall, daily_total, line_curves, is_interchange and all_lines.

Why local import? It avoids importing scheduling_optimizer at module top, but scheduling_optimizer itself loads static files at import. A function call on an interchange therefore imports it if it was not already loaded. The runtime module can tolerate this import because routes imports scheduling too.

Verified naming caveat: docstring says 24-hour profile, but the list has 19 buckets (05 through 23 inclusive). daily_total sums hourly estimates, not minute-level integration. The 23:00 modeled bucket is present even though the service window closes at 23:30; it should not be described as a full 60-minute count after close.

Potential divide-by-zero risk: unknown station fallback has all_lines=[]; is_interchange false but get_station_info returns line Unknown, so the function uses all_lines empty and creates no curves. The fallback expression in get() only applies if all_lines key is missing, not when it exists as an empty list.

7. get_model_evaluation_summary() -> dict

Returns a shallow dictionary expansion of _METRICS plus three runtime keys: synthetic-only basis, real_world_validation_available=False, feature_count from saved feature_cols, artifact_warning=None. Since keys after **_METRICS are written last, these constants override any same-named JSON entries. This method describes stored evaluation; it does not recompute metrics.

8. pipeline/03_train_and_evaluate_forecaster.py - module setup

The script imports joblib/NumPy/pandas and sklearn HistGradientBoostingRegressor, Ridge and metrics. BASE_DIR is inserted into sys.path so importing app.calendar_context works when the script is run directly. It defines synthetic input, small metadata inputs and three artifact outputs. SPLIT_DATE is the string 2026-10-01, compared with pandas datetime values.

Main guard means direct execution runs training; importing the module defines functions/constants without starting model training.

9. prepare_annual_data() - input to model matrix

Read, enrich and encode

• Read synthetic CSV parsing Date into datetime; read cleaned station rows.

• Optionally load typology JSON. Missing file means all stations use the default residential category.

• Select Station, Is_Interchange, Num_Lines and drop duplicate combinations; left-merge into footfall rows. Left join preserves generated demand even if metadata is missing. Missing interchange/line count becomes false/1.

• Create fixed typology2idx dictionary and map station to category; missing type falls back to residential; unknown typology index becomes zero.

• Derive weekday, month, day-of-month, day-of-year, weekend.

• Build contexts once per unique date rather than once per row, then map holiday type, holiday flag, workday. This reduces repeated Python date-context calls.

• Add sin/cos cyclical encodings for hour, weekday and month.

• Sort unique station/line labels and map them to integer indices. These IDs are category labels, not magnitudes with ordinal meaning.

Chronological split

train_mask is Date < SPLIT_DATE. train_df is a copy before split; test_df is all rows on/after split. Summary output prints actual min/max/count. This prevents future rows from contributing to the training fit, but holdout is still synthetic data generated by the same code recipe.

Historical features and leakage controls

Global/station/station-weekday-hour/station-month means are first computed from all training dates. Those maps are used for future test rows and later runtime inference.

For training rows, the code separately constructs prior-only features:

• Sort training rows by date, station index, hour.

• Group by station/date and average hourly Footfall into Daily_Mean.

• For each station's dates, cumulative sum minus current Daily_Mean divided by prior date count yields mean over earlier station-days. Replacing zero count with NaN marks first history as unavailable.

• Repeat per station/month group for prior same-month historical mean.

• Merge these daily prior values back to hourly training rows with many-to-one validation.

• Group hourly training rows by station index, weekday, hour. cumcount is number of prior occurrences in that group; cumulative sum minus current target divided by that prior count forms the prior-only weekday-hour mean.

• Initial unavailable values are NaN; HistGradientBoostingRegressor accepts NaN natively.

• For holdout rows, fill historical features from all training dates only; unseen keys fall back to station mean, then global mean.

This avoids a training row using its own target or later date targets as its aggregate feature. It does not make the synthetic target realistic; it only improves the validity of the split within that synthetic scenario.

Return contract and complexity

Returns train_df, test_df, feature_cols, metadata. Dataset operations are broadly O(N) to O(N log N) due groupings/sort/merge; memory is O(N) because copies and intermediate tables coexist. The generated dataset has 3,153,696 rows, so this offline step is memory-heavy relative to serving.

Metadata includes both-direction station/line mappings, typology maps, calendar maps, historical means, exact feature_cols and split_date. Training saves this metadata with the model so runtime can reproduce the feature contract.

10. compute_metrics(y_true, y_pred)

Calculates MAE, root mean squared error, R-squared and MAPE. MAPE uses only targets where y_true > 0, preventing divide-by-zero but changing its population: it excludes zero-demand targets. Returns rounded Python floats in a dictionary.

Edge: if there are no positive y_true values, np.mean of an empty vector returns NaN with a warning; function does not special-case it. MAE/RMSE/R2 also have undefined/degenerate cases for empty arrays or constant targets. Current metrics file reports a nonempty synthetic holdout.

11. train_and_evaluate_model()

1. Gets prepared split, schema and metadata. 2. Selects feature columns into X_train/X_test, targets into NumPy values. 3. Baseline 1 predicts per-row station historical mean; baseline 2 predicts station-weekday-hour history. These are feature-based naive comparisons. 4. Ridge(alpha=100) trains on X_train.fillna(0); predictions are clipped to nonnegative. Missing history is represented as zero only for Ridge. 5. HistGradientBoostingRegressor uses 250 iterations, learning rate 0.06, max_depth 9, min_samples_leaf 25, l2_regularization 0.05, seed 42. It trains with NaNs retained. 6. Evaluate held-out predictions through compute_metrics. 7. Create models directory if needed, save model then metadata with joblib.dump. 8. Assemble JSON report with row counts/date split, validation description, model comparison and feature names; dump to evaluation_metrics.json; return report. 9. Script main guard prints completion after direct invocation.

Side effects: writes model, metadata and metrics files; prints progress. If training fails after model dump but before metadata/report write, artifacts can become out of sync. A safer publication process writes to a temporary versioned directory and atomically swaps a matched artifact bundle.

12. Evaluation interpretation

The checked-in metrics record a 411,540-row chronological synthetic test set and model MAE 27.90, RMSE 54.82, R\u00b2 0.9947, MAPE 5.16% for HistGradientBoostingRegressor. Baselines have worse values. These figures describe fit to synthetic labels generated under authored rules. They do not establish accuracy on observed DMRC riders. The dataset is not independent of its generator recipe.

13. Interview questions and exercises

Why save feature_cols inside metadata? It defines the exact order used to build the inference DataFrame; column ordering/schema drift can break or corrupt predictions.

How does the code avoid self-target leakage in training history? For each training row, cumulative sum subtracts that row's own value, then divides by the number of earlier group observations. Data is sorted chronologically first.

Why retain NaN history for HistGradientBoosting but fill with zero for Ridge? This estimator handles missing values natively; the linear baseline's numeric design matrix needs a finite representation.

Does the test split prove the model generalizes to Delhi ridership? No; both train and test labels come from this repository's synthetic scenario process.

Exercise: Explain the first row in one station's daily group. Its prior station-day count is zero, replaced with NaN; therefore there is no historical station average available yet. HistGradientBoosting can consume this missing value. Add a test or assertion in a future pipeline that confirms the first historical value per station is missing and later values use only earlier dates.

Proposed improvement: save a manifest with source revision, Python/sklearn versions, input hashes, feature schema and training timestamp alongside artifacts; verify manifest at app startup. No such artifact manifest currently exists.

Source map

Function/block	Purpose	Main consumers
app/forecasting.py:15-32	artifact paths and process-level loading	every inference call
app/forecasting.py:34-53	station row adapter	forecast/routes/profile
app/forecasting.py:56-116	23-feature row and estimator prediction	routes/profile
app/forecasting.py:119-130	minute interpolation + service gate	routes
app/forecasting.py:133-188	19-hour profile + line curves	dashboard/landing/result
app/forecasting.py:191-198	saved evaluation/provenance response	routes
pipeline/03_train_and_evaluate_forecaster.py:48-194	data prep/schema/history split	train entrypoint
pipeline/03_train_and_evaluate_forecaster.py:197-208	score calculation	training
pipeline/03_train_and_evaluate_forecaster.py:211-297	baseline/model/artifact lifecycle	offline pipeline

Appendix A - complete source excerpts

These excerpts come from the documented commit. Original line numbers are printed at left. For print readability, Unicode dash variants are normalized to ASCII hyphens and other non-ASCII source symbols are shown as explicit Unicode escape notation; this does not change the source line numbers or code meaning. Long lines wrap with indented continuations.

Complete runtime forecasting module `app/forecasting.py:1-198`

```python
"""
Passenger Footfall Forecasting Inference Engine.

Wraps the trained machine learning model (HistGradientBoostingRegressor) and
historical baseline models to provide real-time hourly and 24-hour profile predictions.
"""
import os
import joblib
import numpy as np
import pandas as pd
from datetime import datetime
from app.calendar_context import get_calendar_context
from app.operating_hours import is_service_operating

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
MODEL_PATH = os.path.join(BASE_DIR, 'models', 'footfall_forecaster.joblib')
METADATA_PATH = os.path.join(BASE_DIR, 'models', 'forecaster_metadata.joblib')
STATIONS_PATH = os.path.join(BASE_DIR, 'data', 'cleaned', 'stations_cleaned.csv')
EVAL_METRICS_PATH = os.path.join(BASE_DIR, 'models', 'evaluation_metrics.json')

# Load model and metadata
_MODEL = joblib.load(MODEL_PATH)
_META = joblib.load(METADATA_PATH)
_STATIONS_DF = pd.read_csv(STATIONS_PATH).set_index('Station')

# Evaluation metrics
_METRICS = {}
if os.path.exists(EVAL_METRICS_PATH):
    import json
    with open(EVAL_METRICS_PATH, 'r', encoding='utf-8') as f:
        _METRICS = json.load(f)


def get_station_info(station: str) -> dict:
    if station not in _STATIONS_DF.index:
        return {
            "station": station,
            "line": "Unknown",
            "all_lines": [],
            "is_interchange": False,
            "latitude": None,
            "longitude": None
        }
    row = _STATIONS_DF.loc[station]
    all_lines = [l.strip() for l in str(row['All_Lines']).split(',')] if 'All_Lines' in row else [row['Line']]
    return {
        "station": station,
        "line": row['Line'],
        "all_lines": all_lines,
        "is_interchange": bool(row['Is_Interchange']),
        "latitude": float(row['Latitude']) if pd.notna(row['Latitude']) else None,
        "longitude": float(row['Longitude']) if pd.notna(row['Longitude']) else None,
    }


def predict_hourly_footfall(station: str, dt_obj: datetime, hour: int) -> int:
    """
    Predicts footfall for a given station, date, and hour using the trained ML model.
    """
    if hour < 5 or hour > 23:
        return 0
    info = get_station_info(station)
    st_idx = _META['station2idx'].get(station, 0)
    line_idx = _META['line2idx'].get(info['line'], 0)

    day_of_week = dt_obj.weekday()
    is_weekend = 1 if day_of_week >= 5 else 0

    hour_sin = np.sin(2 * np.pi * hour / 24.0)
    hour_cos = np.cos(2 * np.pi * hour / 24.0)
    day_sin = np.sin(2 * np.pi * day_of_week / 7.0)
    day_cos = np.cos(2 * np.pi * day_of_week / 7.0)

    # Historical features
    hist_st_mean = _META['hist_station_mean'].get(st_idx, _META['global_mean'])
    hist_key = f"{st_idx}_{day_of_week}_{hour}"
    hist_hd_mean = _META['hist_hour_day_mean'].get(hist_key, hist_st_mean)

    month = dt_obj.month
    date_context = get_calendar_context(dt_obj.date())
    holiday_type = date_context['holiday_type']
    typology = _META.get('typology_dict', {}).get(station, 'RESIDENTIAL_SUBURB')
    typology_idx = _META.get('typology2idx', {}).get(typology, 0)
    station_month_key = f"{st_idx}_{month}"
    station_month_mean = _META.get('hist_station_month_mean', {}).get(station_month_key, hist_st_mean)
    values = {
        'Hour': hour,
        'DayOfWeek': day_of_week,
        'Month': month,
        'DayOfMonth': dt_obj.day,
        'DayOfYear': dt_obj.timetuple().tm_yday,
        'IsWeekend': is_weekend,
        'IsHoliday': int(bool(holiday_type)),
        'IsWorkday': int(date_context['is_workday']),
        'HolidayType': holiday_type,
        'Hour_Sin': hour_sin,
        'Hour_Cos': hour_cos,
        'Day_Sin': day_sin,
        'Day_Cos': day_cos,
        'Month_Sin': np.sin(2 * np.pi * month / 12.0),
        'Month_Cos': np.cos(2 * np.pi * month / 12.0),
        'Station_Idx': st_idx,
        'Line_Idx': line_idx,
        'Typology_Idx': typology_idx,
        'Is_Interchange': 1 if info['is_interchange'] else 0,
        'Num_Lines': len(info['all_lines']),
        'Hist_Station_Mean': hist_st_mean,
        'Hist_Hour_Day_Mean': hist_hd_mean,
```

```
        'Hist_Station_Month_Mean': station_month_mean,
    }
    # Match the loaded model's schema so inference stays aligned with the
    # persisted training metadata when the feature set changes.
    features = pd.DataFrame([{name: values[name] for name in _META['feature_cols']}])

    pred = _MODEL.predict(features)[0]
    return max(0, round(float(pred)))


def predict_footfall_with_minute_interpolation(station: str, dt_obj: datetime, hour: int, minute: int) -> int:
    """
    Interpolates smoothly across the hour based on the minute.
    """
    if not is_service_operating(hour, minute):
        return 0
    v0 = predict_hourly_footfall(station, dt_obj, hour)
    next_hour = min(23, hour + 1)
    v1 = predict_hourly_footfall(station, dt_obj, next_hour)

    interpolated = v0 + (v1 - v0) * (minute / 60.0)
    return max(0, round(interpolated))


def predict_station_full_day_profile(station: str, dt_obj: datetime) -> dict:
    """
    Generates 24-hour demand profile for charts (05:00 to 23:00).
    """
    hours = list(range(5, 24))
    predicted_curve = []
    historical_curve = []

    st_idx = _META['station2idx'].get(station, 0)
    day_of_week = dt_obj.weekday()

    for h in hours:
        # ML prediction
        pred_val = predict_hourly_footfall(station, dt_obj, h)
        predicted_curve.append(pred_val)

        # Historical baseline mean
        hist_key = f"{st_idx}_{day_of_week}_{h}"
        hist_val = round(_META['hist_hour_day_mean'].get(hist_key, _META['hist_station_mean'].get(st_idx, 0)))
        historical_curve.append(hist_val)

    labels = [f"{h:02d}:00" for h in hours]

    # Identify peak demand periods
    max_idx = int(np.argmax(predicted_curve))
    peak_hour = labels[max_idx]
    peak_footfall = predicted_curve[max_idx]

    # Daily sum
    predicted_daily_total = sum(predicted_curve)

    # Calculate line-wise 24-hour demand curves
    info = get_station_info(station)
    all_lines = info.get('all_lines', [info['line']])
    is_interchange = len(all_lines) > 1

    from app.scheduling_optimizer import get_interchange_line_split
    line_splits = get_interchange_line_split(station, all_lines) if is_interchange else {info['line']: 1.0}

    line_curves = {}
    for l in all_lines:
        share = line_splits.get(l, 1.0 / len(all_lines))
        line_curves[l] = [round(val * share) for val in predicted_curve]

    return {
        "labels": labels,
        "hours": hours,
        "predicted": predicted_curve,
        "historical": historical_curve,
        "peak_hour": peak_hour,
        "peak_footfall": peak_footfall,
        "daily_total": predicted_daily_total,
        "line_curves": line_curves,
        "is_interchange": is_interchange,
        "all_lines": all_lines
    }


def get_model_evaluation_summary() -> dict:
    return {
        **_METRICS,
        "data_basis": "SYNTHETIC_SIMULATION_ONLY",
        "real_world_validation_available": False,
        "feature_count": len(_META['feature_cols']),
        "artifact_warning": None,
    }
```

Complete offline feature preparation and training pipeline

`pipeline/03_train_and_evaluate_forecaster.py:1-298`

```
"""
Pipeline Step 3: Train & Chronologically Evaluate Advanced Footfall Forecaster.

Features:
1. Ingests a 104-week synthetic scenario dataset (2025-01 to 2027-01).
2. Advanced Feature Engineering:
   - Calendar & Holiday: hour/date cycles, weekends, workdays, known occasions, and event classes.
   - Cyclical Trigonometric Encodings: Hour (sin/cos), DayOfWeek (sin/cos), Month (sin/cos).
   - Station Typology: Commercial/Office, Intermodal Gateway, University, Shopping/Leisure, Residential.
   - Station Centrality: Line, Is_Interchange, Num_Lines.
   - Historical Aggregates (strictly computed on Training split to prevent any data leakage):
     * Hist_Station_Mean
     * Hist_Hour_Day_Mean
     * Hist_Station_Month_Mean
3. Chronological Out-of-Time Split:
   - Train through 2026-09-30; test from 2026-10-01 onward, including Dussehra/Diwali.
4. Evaluates:
   - Baseline 1: Overall Station Mean
   - Baseline 2: Historical (Station, DayOfWeek, Hour) Mean
   - Baseline 3: Linear Ridge Regression
   - Production Model: Tuned HistGradientBoostingRegressor
5. Saves model artifacts, metadata, and evaluation report to models/.
"""
import os
import sys
import json
import joblib
from datetime import date
import numpy as np
import pandas as pd
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, BASE_DIR)
from app.calendar_context import get_calendar_context

FOOTFALL_DATA_PATH = os.path.join(BASE_DIR, 'data', 'synthetic', 'dmrc_calibrated_footfall.csv')
STATIONS_CLEANED_PATH = os.path.join(BASE_DIR, 'data', 'cleaned', 'stations_cleaned.csv')
TYPOLOGY_PATH = os.path.join(BASE_DIR, 'data', 'cleaned', 'station_typologies.json')
MODEL_OUT_PATH = os.path.join(BASE_DIR, 'models', 'footfall_forecaster.joblib')
METADATA_OUT_PATH = os.path.join(BASE_DIR, 'models', 'forecaster_metadata.joblib')
METRICS_OUT_PATH = os.path.join(BASE_DIR, 'models', 'evaluation_metrics.json')

SPLIT_DATE = '2026-10-01'  # Hold out the future festival quarter and year end


def prepare_annual_data():
    print(f"Loading 104-week footfall dataset from: {FOOTFALL_DATA_PATH}")
    df = pd.read_csv(FOOTFALL_DATA_PATH, parse_dates=['Date'])
    stations_df = pd.read_csv(STATIONS_CLEANED_PATH)

    # Load typologies
    typology_dict = {}
    if os.path.exists(TYPOLOGY_PATH):
        with open(TYPOLOGY_PATH, 'r', encoding='utf-8') as f:
            typology_dict = json.load(f)

    # Merge station attributes
    station_meta = stations_df[['Station', 'Is_Interchange', 'Num_Lines']].drop_duplicates()
    df = df.merge(station_meta, on='Station', how='left')
    df['Is_Interchange'] = df['Is_Interchange'].fillna(False).astype(int)
    df['Num_Lines'] = df['Num_Lines'].fillna(1).astype(int)

    # Station Typology encoding
    typologies_unique = ["RESIDENTIAL_SUBURB", "COMMERCIAL_OFFICE", "TRANSIT_GATEWAY", "UNIVERSITY_CAMPUS", "SHOPPING_LEISURE"]
    typology2idx = {t: i for i, t in enumerate(typologies_unique)}
    df['Typology'] = df['Station'].map(lambda s: typology_dict.get(s, "RESIDENTIAL_SUBURB"))
    df['Typology_Idx'] = df['Typology'].map(typology2idx).fillna(0).astype(int)

    # Calendar & Holiday features
    df['DayOfWeek'] = df['Date'].dt.weekday
    df['Month'] = df['Date'].dt.month
    df['DayOfMonth'] = df['Date'].dt.day
    df['DayOfYear'] = df['Date'].dt.dayofyear
    df['IsWeekend'] = (df['DayOfWeek'] >= 5).astype(int)

    # Holiday indicators
    date_strs = df['Date'].dt.strftime('%Y-%m-%d')
    contexts = {
        d: get_calendar_context(pd.Timestamp(d).date())
        for d in date_strs.unique()
    }
    holiday_map = {d: ctx['holiday_type'] for d, ctx in contexts.items()}
    public_holiday_map = {d: ctx['is_public_holiday'] for d, ctx in contexts.items()}
    df['HolidayType'] = date_strs.map(holiday_map).fillna(0).astype(int)
    df['IsHoliday'] = (df['HolidayType'] > 0).astype(int)
    df['IsWorkday'] = (
        (df['DayOfWeek'] < 5) & ~date_strs.map(public_holiday_map).fillna(False)
    ).astype(int)

    # Cyclical trigonometric encodings
    df['Hour_Sin'] = np.sin(2 * np.pi * df['Hour'] / 24.0)
    df['Hour_Cos'] = np.cos(2 * np.pi * df['Hour'] / 24.0)
    df['Day_Sin'] = np.sin(2 * np.pi * df['DayOfWeek'] / 7.0)
    df['Day_Cos'] = np.cos(2 * np.pi * df['DayOfWeek'] / 7.0)
    df['Month_Sin'] = np.sin(2 * np.pi * df['Month'] / 12.0)
    df['Month_Cos'] = np.cos(2 * np.pi * df['Month'] / 12.0)

    # Station & Line Categoricals
    stations_unique = sorted(df['Station'].unique().tolist())
    lines_unique = sorted(df['Line'].unique().tolist())

    station2idx = {s: i for i, s in enumerate(stations_unique)}
    line2idx = {l: i for i, l in enumerate(lines_unique)}

```

```
    df['Station_Idx'] = df['Station'].map(station2idx)
    df['Line_Idx'] = df['Line'].map(line2idx)

    # Chronological Out-of-Time Split
    train_mask = df['Date'] < SPLIT_DATE
    train_df = df[train_mask].copy()
    test_df = df[~train_mask].copy()

    print(f"Chronological Out-of-Time Split:")
    print(f"  Training set (before split): {train_df['Date'].min().strftime('%Y-%m-%d')} to {train_df['Date'].max().strftime('%Y-%m-%d')} ({len(train_df):,} rows)")
    print(f"  Future holdout:              {test_df['Date'].min().strftime('%Y-%m-%d')} to {test_df['Date'].max().strftime('%Y-%m-%d')} ({len(test_df):,} rows)")

    # Final training-period aggregates are used for forecasts after the split.
    # Training rows use prior-date-only aggregates below, preventing a row's
    # own target (or later training dates) from leaking into its features.
    global_mean = train_df['Footfall'].mean()
    hist_station_mean = train_df.groupby('Station_Idx')['Footfall'].mean().to_dict()
    hist_hour_day_mean = train_df.groupby(['Station_Idx', 'DayOfWeek', 'Hour'])['Footfall'].mean().to_dict()
    hist_station_month_mean = train_df.groupby(['Station_Idx', 'Month'])['Footfall'].mean().to_dict()

    # Daily station summaries let station and station-month means use only
    # completed earlier dates, with no same-day information in a forecast.
    train_df = train_df.sort_values(['Date', 'Station_Idx', 'Hour']).copy()
    daily = (train_df.groupby(['Station_Idx', 'Date'], as_index=False, sort=True)
             .agg(Daily_Mean=('Footfall', 'mean')))
    daily['Month'] = pd.to_datetime(daily['Date']).dt.month
    station_group = daily.groupby('Station_Idx', sort=False)['Daily_Mean']
    daily['Hist_Station_Mean'] = (
        (station_group.cumsum() - daily['Daily_Mean']) /
        station_group.cumcount().replace(0, np.nan)
    )
    month_group = daily.groupby(['Station_Idx', 'Month'], sort=False)['Daily_Mean']
    daily['Hist_Station_Month_Mean'] = (
        (month_group.cumsum() - daily['Daily_Mean']) /
        month_group.cumcount().replace(0, np.nan)
    )
    train_df = train_df.merge(
        daily[['Station_Idx', 'Date', 'Hist_Station_Mean', 'Hist_Station_Month_Mean']],
        on=['Station_Idx', 'Date'], how='left', validate='many_to_one'
    )

    hour_day_keys = ['Station_Idx', 'DayOfWeek', 'Hour']
    hour_day_group = train_df.groupby(hour_day_keys, sort=False)['Footfall']
    prior_hour_day_count = hour_day_group.cumcount()
    train_df['Hist_Hour_Day_Mean'] = (
        (hour_day_group.cumsum() - train_df['Footfall']) /
        prior_hour_day_count.replace(0, np.nan)
    )

    # Use all training dates only for test rows and live inference.
    test_df['Hist_Station_Mean'] = test_df['Station_Idx'].map(hist_station_mean)
    test_df['Hist_Hour_Day_Mean'] = [
        hist_hour_day_mean.get((station, day, hour), hist_station_mean.get(station, global_mean))
        for station, day, hour in zip(test_df['Station_Idx'], test_df['DayOfWeek'], test_df['Hour'])
    ]
    test_df['Hist_Station_Month_Mean'] = [
        hist_station_month_mean.get((station, month), hist_station_mean.get(station, global_mean))
        for station, month in zip(test_df['Station_Idx'], test_df['Month'])
    ]
    # Initial history is genuinely unavailable for early training rows.
    # HistGradientBoostingRegressor handles missing history values natively.

    feature_cols = [
        'Hour', 'DayOfWeek', 'Month', 'DayOfMonth', 'DayOfYear',
        'IsWeekend', 'IsHoliday', 'IsWorkday', 'HolidayType',
        'Hour_Sin', 'Hour_Cos', 'Day_Sin', 'Day_Cos', 'Month_Sin', 'Month_Cos',
        'Station_Idx', 'Line_Idx', 'Typology_Idx', 'Is_Interchange', 'Num_Lines',
        'Hist_Station_Mean', 'Hist_Hour_Day_Mean', 'Hist_Station_Month_Mean'
    ]

    metadata = {
        'station2idx': station2idx,
        'idx2station': {i: s for s, i in station2idx.items()},
        'line2idx': line2idx,
        'idx2line': {i: l for l, i in line2idx.items()},
        'typology2idx': typology2idx,
        'typology_dict': typology_dict,
        'delhi_holidays': {k: ctx['occasion'] for k, ctx in contexts.items() if ctx['occasion']},
        'holiday_type_by_date': holiday_map,
        'hist_station_mean': hist_station_mean,
        'hist_hour_day_mean': {f"{k[0]}_{k[1]}_{k[2]}": v for k, v in hist_hour_day_mean.items()},
        'hist_station_month_mean': {f"{k[0]}_{k[1]}": v for k, v in hist_station_month_mean.items()},
        'global_mean': global_mean,
        'feature_cols': feature_cols,
        'split_date': SPLIT_DATE
    }

    return train_df, test_df, feature_cols, metadata


def compute_metrics(y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = root_mean_squared_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    non_zero = y_true > 0
    mape = np.mean(np.abs((y_true[non_zero] - y_pred[non_zero]) / y_true[non_zero])) * 100.0
    return {
        'MAE': round(float(mae), 2),
        'RMSE': round(float(rmse), 2),
        'R2': round(float(r2), 4),
        'MAPE_%': round(float(mape), 2)
    }


def train_and_evaluate_model():
    train_df, test_df, feature_cols, metadata = prepare_annual_data()

    X_train = train_df[feature_cols]
    y_train = train_df['Footfall'].values
```

```
    X_test = test_df[feature_cols]
    y_test = test_df['Footfall'].values

    print("\n--- Evaluating Baseline 1: Station Overall Mean ---")
    b1_pred = test_df['Hist_Station_Mean'].values
    m_b1 = compute_metrics(y_test, b1_pred)
    print(f"  Station Mean Baseline: {m_b1}")

    print("\n--- Evaluating Baseline 2: Historical (Station, Day, Hour) Mean ---")
    b2_pred = test_df['Hist_Hour_Day_Mean'].values
    m_b2 = compute_metrics(y_test, b2_pred)
    print(f"  Station-Day-Hour Mean Baseline: {m_b2}")

    print("\n--- Evaluating Baseline 3: Ridge Regression ---")
    ridge = Ridge(alpha=100.0)
    ridge.fit(X_train.fillna(0), y_train)
    ridge_pred = np.clip(ridge.predict(X_test), 0, None)
    m_ridge = compute_metrics(y_test, ridge_pred)
    print(f"  Ridge Regression: {m_ridge}")

    print("\n--- Training Production Model: Tuned HistGradientBoostingRegressor ---")
    model = HistGradientBoostingRegressor(
        max_iter=250,
        learning_rate=0.06,
        max_depth=9,
        min_samples_leaf=25,
        l2_regularization=0.05,
        random_state=42
    )
    model.fit(X_train, y_train)

    ml_pred = np.clip(model.predict(X_test), 0, None)
    m_ml = compute_metrics(y_test, ml_pred)
    print(f"  Tuned HistGradientBoostingRegressor: {m_ml}")

    # Save model and metadata
    os.makedirs(os.path.dirname(MODEL_OUT_PATH), exist_ok=True)
    joblib.dump(model, MODEL_OUT_PATH)
    joblib.dump(metadata, METADATA_OUT_PATH)
    print(f"\nModel saved to: {MODEL_OUT_PATH}")
    print(f"Metadata saved to: {METADATA_OUT_PATH}")

    # Save comprehensive metrics report
    report = {
        "evaluation_dataset": {
            "name": "Delhi Metro Synthetic Scenario Test Split",
            "total_records": len(train_df) + len(test_df),
            "train_samples": len(train_df),
            "test_samples": len(test_df),
            "split_date": SPLIT_DATE,
            "validation_method": "Chronological Out-of-Time Validation (future festival quarter; prior-date history features)"
        },
        "model_comparison": {
            "baseline_station_mean": {
                "name": "Station Overall Mean Baseline",
                "metrics": m_b1
            },
            "baseline_historical_hour_day": {
                "name": "Historical (Station, DayOfWeek, Hour) Mean Baseline",
                "metrics": m_b2
            },
            "baseline_ridge_regression": {
                "name": "Ridge Regression Baseline",
                "metrics": m_ridge
            },
            "hist_gradient_boosting": {
                "name": "Tuned HistGradientBoostingRegressor (Production ML)",
                "metrics": m_ml
            }
        },
        "feature_importance_features": feature_cols
    }

    with open(METRICS_OUT_PATH, 'w', encoding='utf-8') as f:
        json.dump(report, f, indent=2)
    print(f"Metrics report saved to: {METRICS_OUT_PATH}")
    return report


if __name__ == "__main__":
    train_and_evaluate_model()
    print("Step 3 Advanced Model Training & Evaluation completed successfully!")
```